



Санкт-Петербургский государственный университет
Кафедра системного программирования

Теория графов. Презентации 1-2

Команда 7: Павлушкин Дмитрий (Шишин Кирилл, Елкин Леонид)

Санкт-Петербург
2025

PageRank, постановка задачи

Дан невзвешенный ориентированный граф $\mathcal{G} = (\mathcal{V}, \mathcal{E})$. Необходимо найти "важность" каждой вершины в графе.

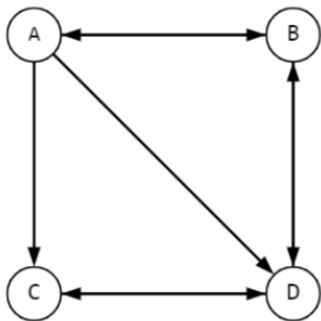
В классической версии алгоритма изначально "важность" каждой вершины равна $\frac{1}{|\mathcal{V}|}$. На каждой новой итерации алгоритма "важность" каждой пересчитывается в соответствии с формулой

$$PR(u) = \frac{1-d}{|\mathcal{V}|} + d \sum_{v \in B(u)} \frac{PR(v)}{L(v)}, \text{ где}$$

- $B(u)$ — множество вершин, из которых есть путь в вершину u
- $L(v)$ — количество путей, исходящих из вершины v
- d — коэффициент затухания (обычно 0.85)

Пересчет "важности" вершин производится до сходимости.

$$L = \begin{pmatrix} 0 & \frac{1}{2} & 0 & 0 \\ \frac{1}{3} & 0 & 0 & \frac{1}{2} \\ \frac{1}{3} & 0 & 0 & \frac{1}{2} \\ \frac{1}{3} & \frac{1}{2} & 1 & 0 \end{pmatrix}$$



$$M = (dL + (1 - d)E)$$

Где E – матрица из $\frac{1}{N}$

$$PR^{(t+1)} = M * PR^{(t)}$$

Сходимость Pagerank

- Матрица переходов M стохастическая (сумма элементов в столбцах равна 1)
- Элементы этой матрицы строго положительные

Из Т. Перрона-Фробениуса следует¹

- M имеет уникальный собственный вектор, который соответствует максимальному собственному числу $\lambda=1$. (Этот же вектор является стационарным)
- $PR^{(t+1)} = M * PR^{(t)}$ сходится к этому собственному вектору.

¹Согласно материалам статьи

Определение (Описание)

Apache Spark — движок для **распределённой** обработки данных. Позволяет обрабатывать большие объёмы данных, которые не помещаются в память одного компьютера. Работает с кластерами (несколько серверов).

Определение (Основные компоненты)

- *GraphX* – графовые данные.
- *Spark SQL* – работа с таблицами и SQL-запросами.
- *Spark Streaming* – потоковая обработка данных.
- *MLlib* – машинное обучение.

PySpark и Hadoop

Определение (PySpark)

Spark написан на Scala, а PySpark позволяет писать код на Python, который затем выполняется в Spark.

Определение (Hadoop)

*PySpark может использовать нативные библиотеки Hadoop для ускорения некоторых операций, но их наличие **не обязательно**. Spark полностью работает и без Hadoop, используя встроенные Java-классы.*

Нативные Hadoop-библиотеки ускоряют операции, например:

- сжатие/распаковка данных (Snappy, LZO),
- работа с файловой системой HDFS,
- оптимизация чтения/записи данных.

PySpark — особенности распределённой обработки

- Архитектура Driver и Executors:
 - ▶ **Driver** — центральный узел, который координирует выполнение задач.
 - ▶ **Executors** — рабочие процессы на узлах кластера, выполняющие задачи. Код и данные передаются по сети, а задачи автоматически распределяются между кластерами.
- Гибкое управление кластером:
 - ▶ Можно создавать локальные или распределённые кластеры.
 - ▶ Поддерживается сериализация данных для передачи между узлами.
 - ▶ Контроль того, где и сколько хранить данных.
- Модель отложенных вычислений (lazy evaluation):
 - ▶ Трансформации, такие как *map()*, *reduce()*, *groupBy()*, не выполняются сразу.
 - ▶ Spark строит граф зависимостей задач и оптимизирует его перед выполнением.
 - ▶ Вычисления запускаются только при вызове действий (actions), *collect()*, *count()*, *saveAs()*.

Основные модули PySpark

В рамках работы мы будем использовать несколько основных модулей PySpark:

- **RDD (Resilient Distributed Dataset)** — базовая абстракция для распределённой обработки данных. Позволяет работать с коллекциями данных через трансформации и действия.
- **Spark SQL / DataFrame** — для удобной работы с таблицами данных, фильтрации, агрегации и SQL-запросов.
- **GraphX (встроенные алгоритмы)** — стандартные алгоритмы обработки графов.

Графы для экспериментов были взяты из Stanford Large Network Dataset Collection. Такой выбор был сделан в силу:

- Представлен широкий выбор графов, которые подходят в том числе для задач поиска кратчайшего пути
- Реалистичность и практическая значимость
- Разнообразие размеров
- Графы SNAP часто используются в публикациях

Набор данных (Directed graphs)

Name	Description	Nodes	Edges
WikiVote	Wikipedia who-votes-on-whom network	7,115	103,689
P2PGnutella	Gnutella peer to peer network	62,586	147,892
Email-EuAll	Email network from a EU research institution	265,214	420,045
Amazon	Amazon product co-purchasing network	262,111	1,234,877
NotreDame	Web graph of Notre Dame	325,729	1,497,134

Цели:

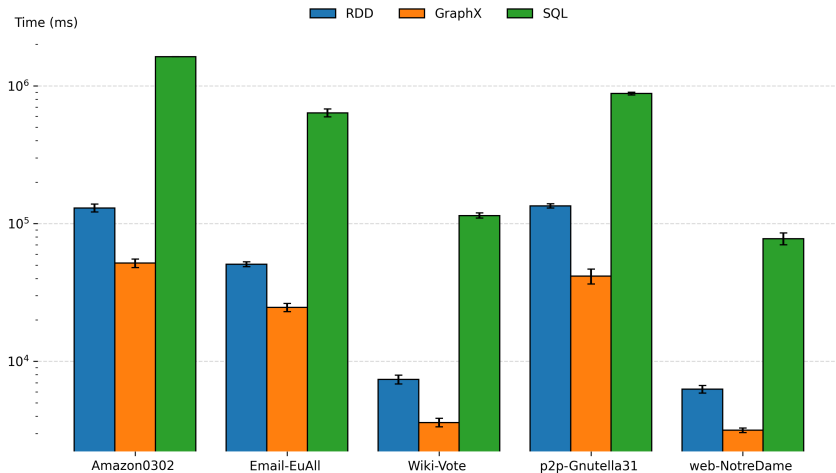
- 1 Оценить производительность распределённых вычислений в PySpark по сравнению с высокопроизводительными реализациями на GPU (Gunrock) и на CPU (GraphBLAS) при обработке небольших наборов данных (менее 5 ГБ).
- 2 Провести сравнение скорости выполнения алгоритма PageRank при использовании разных модулей PySpark, включая RDD, DataFrame/SQL и GraphX.

Каждый вариант будет запускаться по 20 раз

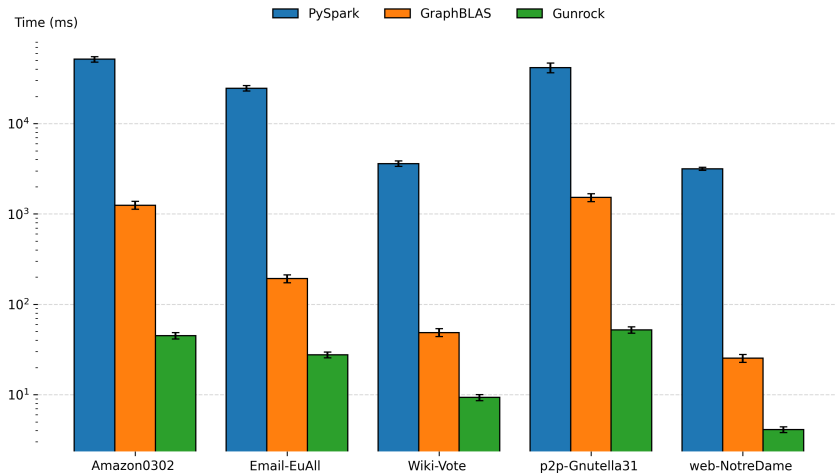
Характеристики вычислительной машины:

- 11th Gen Intel(R) Core(TM) i7-11800H @ 2.30GHz 2.30 GHz
 - ▶ 8 ядер, 16 логических процессоров
 - ▶ Кэш L1 — 640 КБ
 - ▶ Кэш L2 — 10 МБ
 - ▶ Кэш L3 — 24 МБ
- RAM: 32 GB
- GPU: NVIDIA GeForce RTX 3070 laptop
- OS: Windows 11 24H2

Pagerank (Сравнение модулей PySpark)



Pagerank (Сравнение библиотек)



Single-source parent BFS, постановка задачи

Дан невзвешенный неориентированный граф $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, в котором выделена исходная вершина $s \in \mathcal{V}$. Для каждой вершины $v \in \mathcal{V}$ определить вершину $u \in \mathcal{V}$, являющуюся предшественником вершины v на одном из кратчайших путей из s .

Определение (Путь)

Путь $\mathcal{P}$ из вершины s в вершину d — это такая последовательность вершин $v_1, v_2, \dots, v_n$, что $v_1 = s$ и $v_n = d$ и $(v_i, v_{i+1}) \in \mathcal{E}$.

Определение (Кратчайший путь)

$\mathcal{P}^*$ — кратчайший путь из вершины s в вершину d , если $|\mathcal{P}^*| = \min\{|\mathcal{P}| : \mathcal{P} \text{ — путь из } s \text{ в } d\}$

Multi-source parent BFS, постановка задачи

Дан невзвешенный неориентированный граф $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, в котором выделено множество исходных вершин $S \in \mathcal{V}$. Для каждой исходной вершины $s \in S$ и каждой вершины $v \in \mathcal{V}$ определить вершину $u \in \mathcal{V}$, являющуюся предшественником вершины v на одном из кратчайших путей из s .

Набор данных (Undirected graphs)

Name	Description	Nodes	Edges
DeezerEurope	Social network of Deezer users from Europe.	28,281	92,752
CAHepPh	Collaboration network of Arxiv High Energy Physics	12,008	118,521
Enron	Email communication network from Enron	36,692	183,831
Brightkite	Brightkite location based online social network	58,228	214,078
Twitch	Social networks of Twitch users.	34,118	429,113

Эксперименты (SSPBFS и MSPBFS)

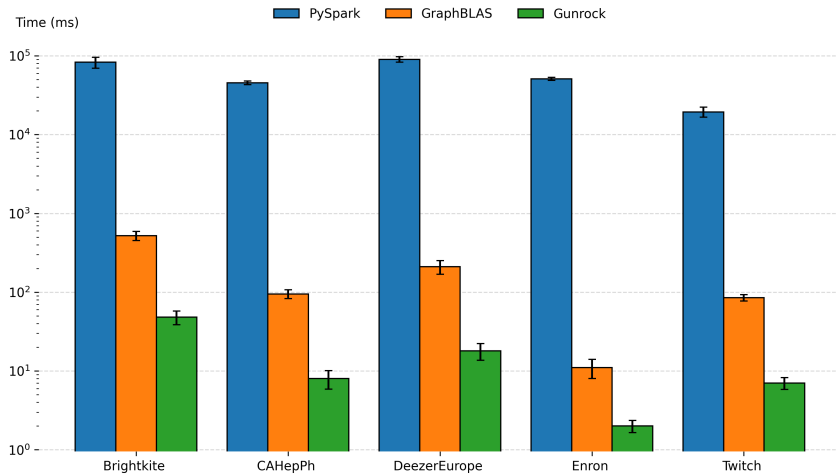
По сравнению с PageRank для SSPBFS и MSPBFS в GraphX нет встроенного алгоритма. Написать реализацию на модуле RDD, влезающую в рамки завершения работы до одного часа, не получилось, поэтому SSPBFS и MSPBFS в PySpark будет реализован только с помощью модуля SQL.

Цели:

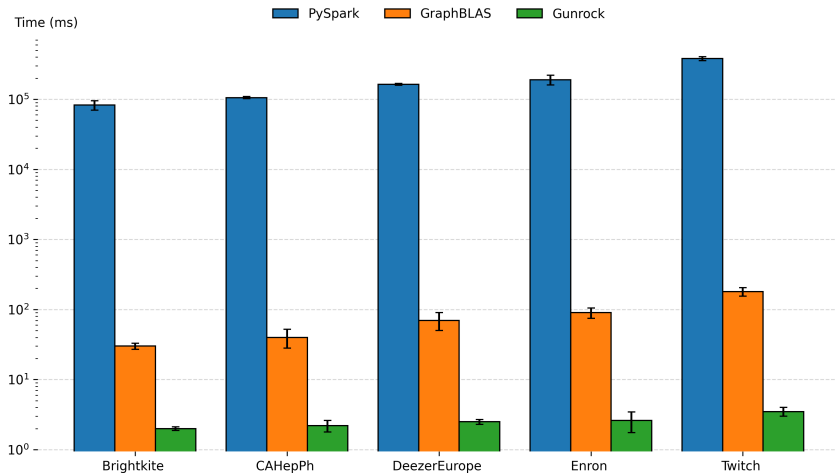
- Оценить производительность распределённых вычислений в PySpark по сравнению с высокопроизводительными реализациями на GPU (Gunrock) и на CPU (GraphBLAS) при обработке небольших наборов данных (менее 5 ГБ).
- Проанализировать на сколько растёт время MSBFS в пределах одной библиотеки с разным количеством начальных вершин относительно SSBFS.

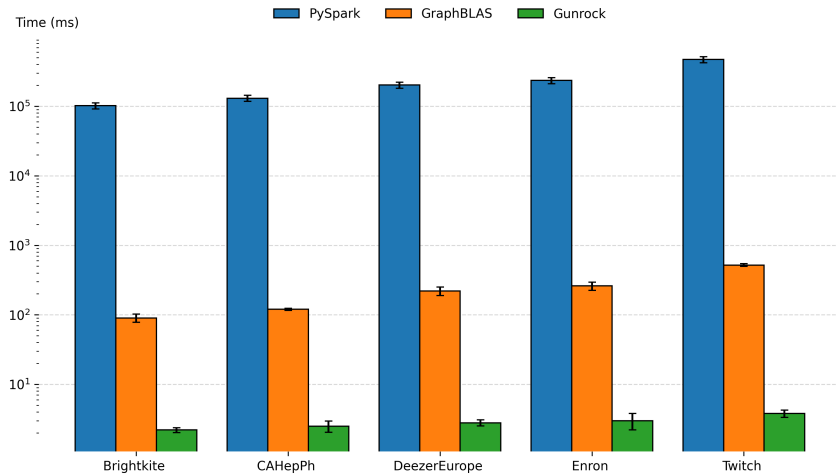
Измерения:

- Для SSPBFS генерировались по 10 случайных вершин из каждого графа и на этих вершинах запускались алгоритмы по 10 раз, брался средний результат.
- Для MSPBFS генерировались 16, 32, 64 случайные вершины из каждого графа, на этих вершинах 10 раз запускался алгоритм.

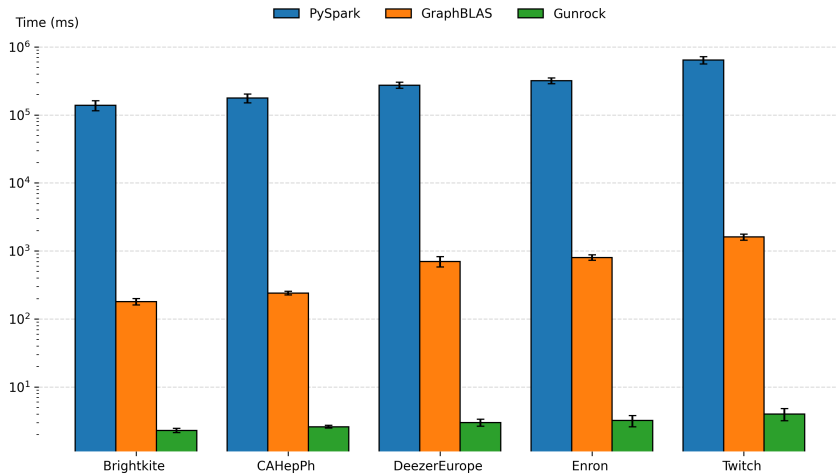


MSPBFS16

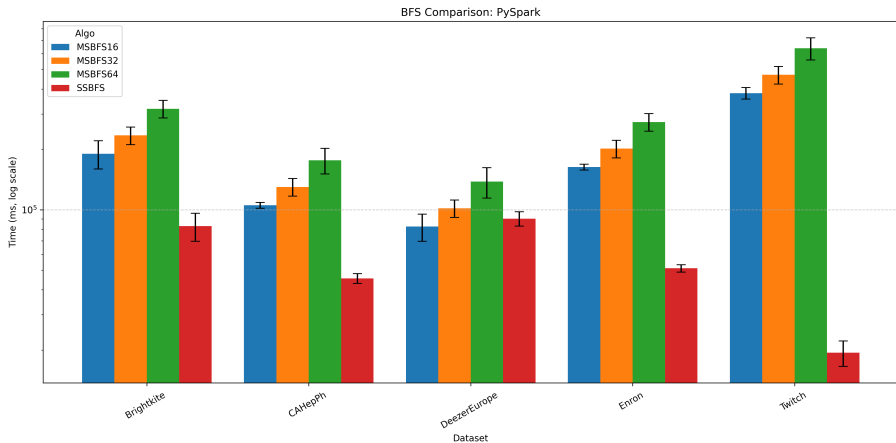




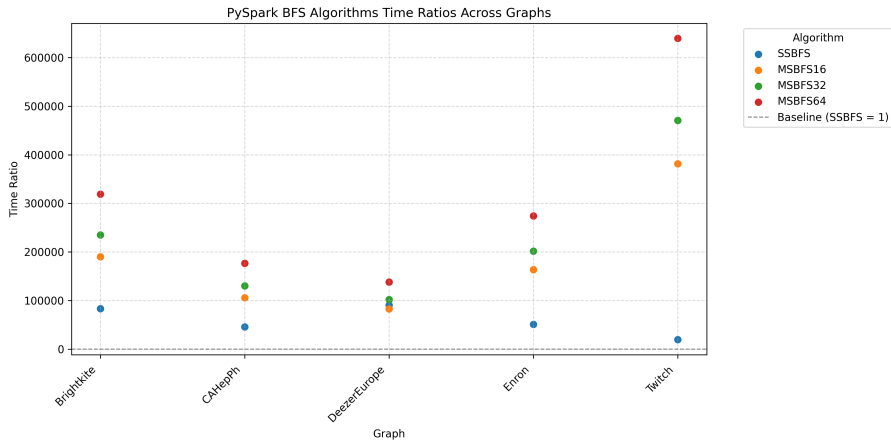
MSPBFS64



PySpark - сравнение разных BFS (1)



PySpark - сравнение разных BFS (2)



Особенности репозитория

- Для каждого алгоритма и используемой библиотеки подготовлены отдельные Docker-образы
- Общий модуль `bench.py` и класс `algo` обеспечивают простую интеграцию алгоритмов в проект
- Универсальный модуль `Visualizer` позволяет строить любые графики, представленные в презентации
- Модуль `Profiler.py` предназначен для анализа логов и оценки временных затрат при выполнении алгоритмов для PySpark